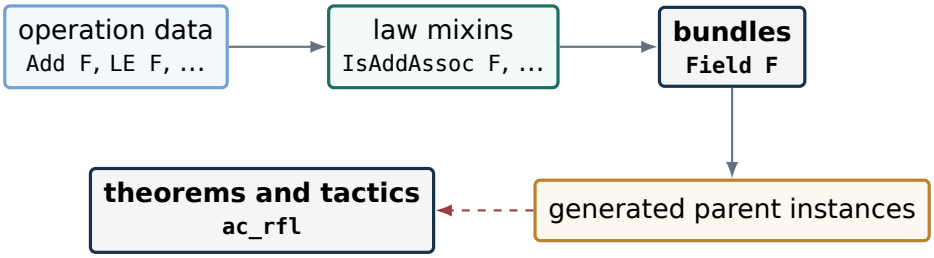
Reading `Numbers.lean`

A visual guide to Lean syntax, mixins, structures, construction, and instance inference



Adventures in Mathematics

Detailed companion to `Aim/Analysis/Numbers.lean`

Code snapshot: 9 November 2026

mathlib-free

Contents

1 What this file is building **3**

2 The outer shell: comments, namespace, and universes **3**
2.1 Three kinds of comments . . . 3
2.2 Namespaces qualify names . . . 4
2.3 Universe variables classify types . . . 4

3 Binders: the punctuation that controls elaboration **4**

4 Data typeclasses and overloaded notation **5**

5 Propositions, proofs, and logical syntax **6**

6 Mixin classes: one reusable pattern **6**
6.1 The archetype . . . 6
6.2 The complete mixin catalog . . . 7

7 Instances: rules in a type-directed search program **7**
7.1 Anonymous functions and operator sections . . . 8
7.2 Anonymous constructor notation . . . 8
7.3 Bridge graph . . . 8

8 Structure extension and generated parent instances **8**
8.1 What `extends` means here . . . 8
8.2 Why no explicit `where` block? . . . 9
8.3 The full hierarchy . . . 9

9 Derived data instances: nonemptiness, subtraction, division **10**
9.1 Definitional equality and `rfl` . . . 10

10 Predicates as sets and order completeness **10**

11 Subtypes: values carrying proofs **11**

12 Recursive construction by pattern matching: `natCast` **12**

13 A dependent mixin: the Archimedean property **12**

14 Ordinary structures: the revised `Distance` **13**

15 Theorem declarations and proof styles **13**
15.1 Direct term proof by projection . . . 13
15.2 Tactic proof . . . 14
15.3 Definitional proof . . . 14

16 End-to-end inference traces **14**
16.1 How `field_smoke_test` finds associativity . . . 14
16.2 How `ac_rfl` crosses the custom/core boundary . . . 14
16.3 How `x < natCast n` elaborates . . . 15

17 Construction: what a future concrete field must provide **15**

18 Syntax index for the entire file **15**

19 A disciplined way to read similar Lean files **16**

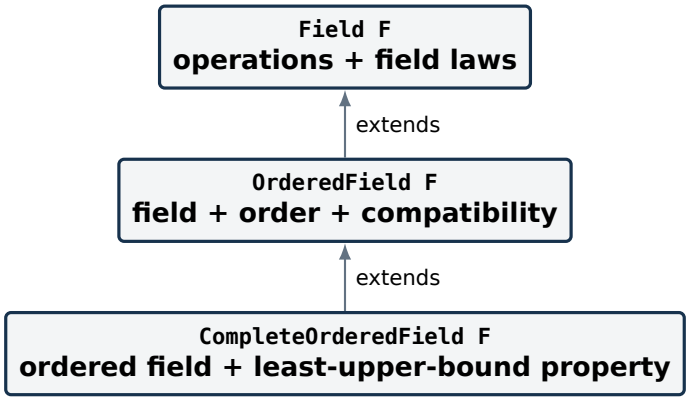
20 Compact glossary **17**

1 What this file is building

The file does not construct the natural numbers, rationals, or reals. It specifies the *interfaces* that later constructions must satisfy. An interface has two kinds of content:

1. **data**: operations and relations such as addition, zero, inverse, and order;
2. **laws**: propositions saying how that data behaves, such as associativity.

The file keeps individual laws in small proposition-valued classes called *mixins*, then combines data and laws into progressively stronger classes:



Big idea

Read `[Field F]` as: "Lean may automatically search for a complete package of field operations and proofs for the type `F`." Once found, the parent packages inside it are also available to instance inference.

The remaining declarations perform four supporting jobs: define useful subtypes, define predicates for least upper bounds, embed core `Nat` values by recursion, and package a nonnegative distance function.

2 The outer shell: comments, namespace, and universes

2.1 Three kinds of comments

```
/- ordinary block comment -/
/-! module documentation -/
/-- declaration documentation -/
-- line comment
```

Ordinary comments are ignored. A module doc comment beginning `/-!` is associated with the file/module. A declaration doc comment beginning `/--` is attached to the next declaration and can be shown by editor tooling. The roadmap and design prose are therefore part of the browsable API documentation, not merely visual separators.

2.2 Namespaces qualify names

```
namespace aim.analysis
  -- Field is really aim.analysis.Field here.
end aim.analysis
```

A namespace prevents short names such as `Field` from occupying the global name space. Inside the namespace, the short name is normally sufficient. Outside, a client can write `aim.analysis.Field`; an `open` command could make short names available without changing their true full names.

2.3 Universe variables classify types

```
universe u v
```

`Type u` is the universe of types at level `u`. Universe levels prevent a "type of all types" paradox while retaining polymorphism. Most declarations use one carrier `X : Type u`. The distance structure uses two levels because the point type and value type need not inhabit the same universe:

```
structure Distance (X : Type u) (R : Type v) ...
```

Lean computes an appropriate universe for `Distance X R`; the author does not have to force `X` and `R` into a common level.

3 Binders: the punctuation that controls elaboration

Nearly every declaration is a dependent function type. The shape of a binder tells Lean how callers will supply its argument.

Syntax	Meaning	How supplied
`(X : Type u)`	explicit parameter	caller normally writes it
`{F : Type u}`	implicit parameter	elaborator infers it from context
`[Add X]`	instance-implicit parameter	typeclass search must synthesize it
`(a b c : F)`	several explicit parameters of one type	caller writes values

For example:

```
theorem field_smoke_test {F : Type u} [Field F] (a b c : F) : ...
```

- `F` is inferred from `a`, `b`, and `c`.
- the `Field F` object is found by instance inference;
- the three elements are ordinary explicit arguments.

Watch this

Square brackets do not mean "optional." They mean "ask the instance synthesizer." Elaboration fails if no appropriate instance can be built, or if search becomes ambiguous or cyclic.

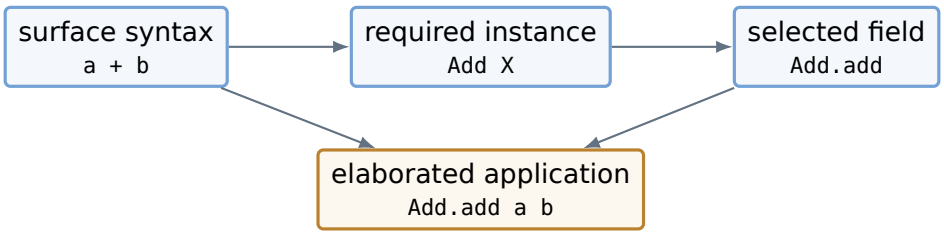
4 Data typeclasses and overloaded notation

Lean notation such as `a + b` is not primitive arithmetic. It is syntax whose meaning depends on a typeclass instance. Conceptually, core classes have fields like:

```
class Add (X : Type u) where
  add : X → X → X

class Zero (X : Type u) where
  zero : X
```

Thus `[Add X]` supplies a binary operation, and `[Zero X]` supplies a chosen element. The notations are elaborated using the carrier type:



The same pattern explains all operation and relation parents in the file:

Class	Notation	Data supplied
`Zero F`	`0`	distinguished zero
`One F`	`1`	distinguished one
`Add F`	`a + b`	binary addition
`Mul F`	`a * b`	binary multiplication
`Neg F`	`-a`	unary negation
`Inv F`	a^{-1}	unary reciprocal
`LE F`	$a \le b$	non-strict relation
`LT F`	$a < b$	strict relation
`Sub F`	`a - b`	binary subtraction
`Div F`	`a / b`	binary division

Numerals are type-directed too. The annotation in `(0 : X) ≠ 1` fixes the type of zero; equality then forces one to have the same type. Elsewhere the surrounding operation or relation often supplies enough expected type information.

5 Propositions, proofs, and logical syntax

In Lean, propositions are types in `Prop`; proofs are values of those types. A class declared `: Prop` contains proof data but no computationally relevant mathematical carrier data.

```
class IsAddAssoc (X : Type u) [Add X] : Prop where
  add_assoc : ∀ a b c : X, a + b + c = a + (b + c)
```

This declaration introduces:

1. a proposition family `IsAddAssoc X`, dependent on a chosen `Add X`;
2. a constructor that builds the class from a proof of the field;
3. a projection `IsAddAssoc.add_assoc` that extracts the proof;
4. typeclass metadata because `class`, rather than `structure`, was used.

The distinct logical forms appearing across the file are:

Form	Reading
`∀ a : X, P a`	for every `a`, produce a proof of `P a`
`P → Q`	a function taking a proof of `P` to a proof of `Q`
`a = b`	propositional equality
`¬ P`	abbreviation for `P → False`
`P ∧ Q`	a pair containing proofs of both propositions
`P ∨ Q`	a tagged choice containing a proof of one side
`P ↔ Q`	implications in both directions
`∃ x, P x`	a witness `x` paired with a proof of `P x`

The arrow therefore has two related readings. In `X → X → X` it is an ordinary function arrow. In `a ≠ 0 → a * a⁻¹ = 1`, both endpoints are propositions, so the function consumes one proof and produces another. This is the propositions-as-types principle in action.

6 Mixin classes: one reusable pattern

6.1 The archetype

Associativity is the archetypal mixin:

```
class IsAddAssoc (X : Type u) [Add X] : Prop where
  add_assoc : ∀ a b c : X, a + b + c = a + (b + c)
```

It says: after a type `X` and a particular addition operation have been chosen, there may be a canonical proof that this operation is associative. The operation is a dependency, not a field of the mixin. Consequently, two different `Add X` instances would define two different elaborated associativity claims.

Mental model

A data class answers “what operation do we use?” A law mixin answers “what can we prove about the operation already selected?” A bundle such as `Field F` stores both

answers together and publishes them to instance inference.

6.2 The complete mixin catalog

The other mixins repeat the same declaration mechanism. They need not each be relearned; only their dependencies and propositions change.

Mixin	Required data	Law stored
`IsAddAssoc X`	`Add X`	addition associates
`IsAddComm X`	`Add X`	addition commutes
`IsMulAssoc X`	`Mul X`	multiplication associates
`IsMulComm X`	`Mul X`	multiplication commutes
`IsZeroNeOne X`	`Zero X, One X`	the identities are distinct
`IsZeroAdd X`	`Zero X, Add X`	zero is a left additive identity
`IsAddNeg X`	`Zero X, Add X, Neg X`	each element plus its negation is zero
`IsOneMul X`	`One X, Mul X`	one is a left multiplicative identity
`IsMulInvCancel X`	`Zero X, One X, Mul X, Inv X`	nonzero elements cancel with inverses
`IsMulAdd X`	`Add X, Mul X`	left distributivity
`IsLERefl X`	`LE X`	reflexivity
`IsLETrans X`	`LE X`	transitivity
`IsLEAntisymm X`	`LE X`	antisymmetry
`IsLETotal X`	`LE X`	totality
`IsLTIffLENotLE X`	`LE X, LT X`	strict order is ≤ plus failure of reverse ≤
`IsAddLEAddRight X`	`Add X, LE X`	right translation preserves order
`IsMulPos X`	`Zero X, Mul X, LT X`	positive times positive is positive
`IsOrderComplete F`	`LE F`	every inhabited, bounded-above predicate has a least upper bound
`ArchimedeanProperty F`	`OrderedField F`	every value lies below a natural-number cast

One-sided laws are deliberate. Commutativity later permits derivation of right identity, the opposite inverse orientation, and right distributivity. The file stores a compact axiom basis rather than every convenient theorem.

7 Instances: rules in a type-directed search program

An `instance` declaration registers a term as a rule available to typeclass inference. Consider the associativity bridge:

```
instance {X : Type u} [Add X] [IsAddAssoc X] :
    Std.Associative ((· + ·) : X → X → X) :=
  ⟨IsAddAssoc.⟩add_assoc
```

Read it from right to left conceptually:

1. Goal: build `Std.Associative ((· + ·) : X → X → X)`.
2. Preconditions: synthesize `Add X` and `IsAddAssoc X`.
3. Extract the custom proof with `IsAddAssoc.add_assoc`.
4. Put that proof into the one-field standard class using `⟨...⟩`.

7.1 Anonymous functions and operator sections

`((· + ·) : X → X → X)` is a section using centered-dot placeholders. It means the same function as `fun a b : X => a + b`. The type annotation fixes both placeholder types and prevents ambiguity.

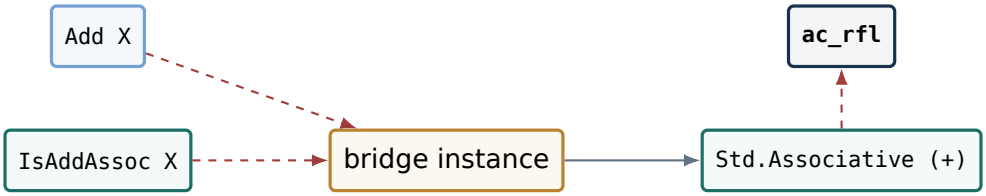
7.2 Anonymous constructor notation

Angle brackets tell Lean to use the expected structure or inductive constructor:

```
⟨
IsAddAssoc.⟩add_assoc⟨⟩
0⟨
fun a b => a + -⟩b
```

The first builds a standard associativity witness, the second builds `Nonempty F`, and the third builds `Sub F`. The expected result type determines which constructor the brackets mean.

7.3 Bridge graph



Four declarations instantiate this pattern: addition/multiplication crossed with associativity/commutativity. The bridge avoids coupling the custom educational hierarchy to a reimplementation of core tactics.

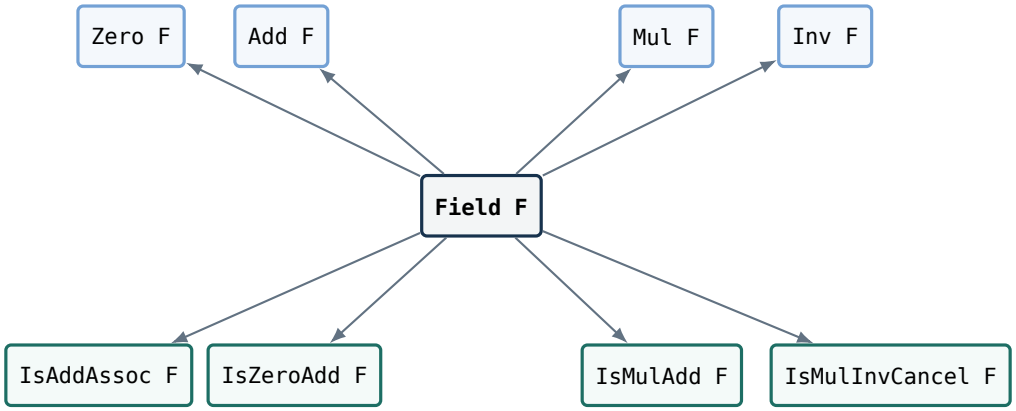
8 Structure extension and generated parent instances

8.1 What `extends` means here

```
class Field (F : Type u)
    extends Zero F, One F, Add F, Mul F, Neg F, Inv F,
      IsZeroNeOne F,
      IsAddAssoc F, IsAddComm F, IsZeroAdd F, IsAddNeg F,
      IsMulAssoc F, IsMulComm F, IsOneMul F, IsMulInvCancel F,
      IsMulAdd F
```

`Field F` is a class structure whose parents are all the listed classes. An object of this type contains the data and proof content needed for every parent. Because the parents are classes, Lean generates projection instances that can recover them.

The ordering matters during elaboration. The data parents occur first. By the time Lean elaborates `IsAddAssoc F`, it can use the inherited `Add F`; similarly, the inverse law can use inherited zero, one, multiplication, and inverse.

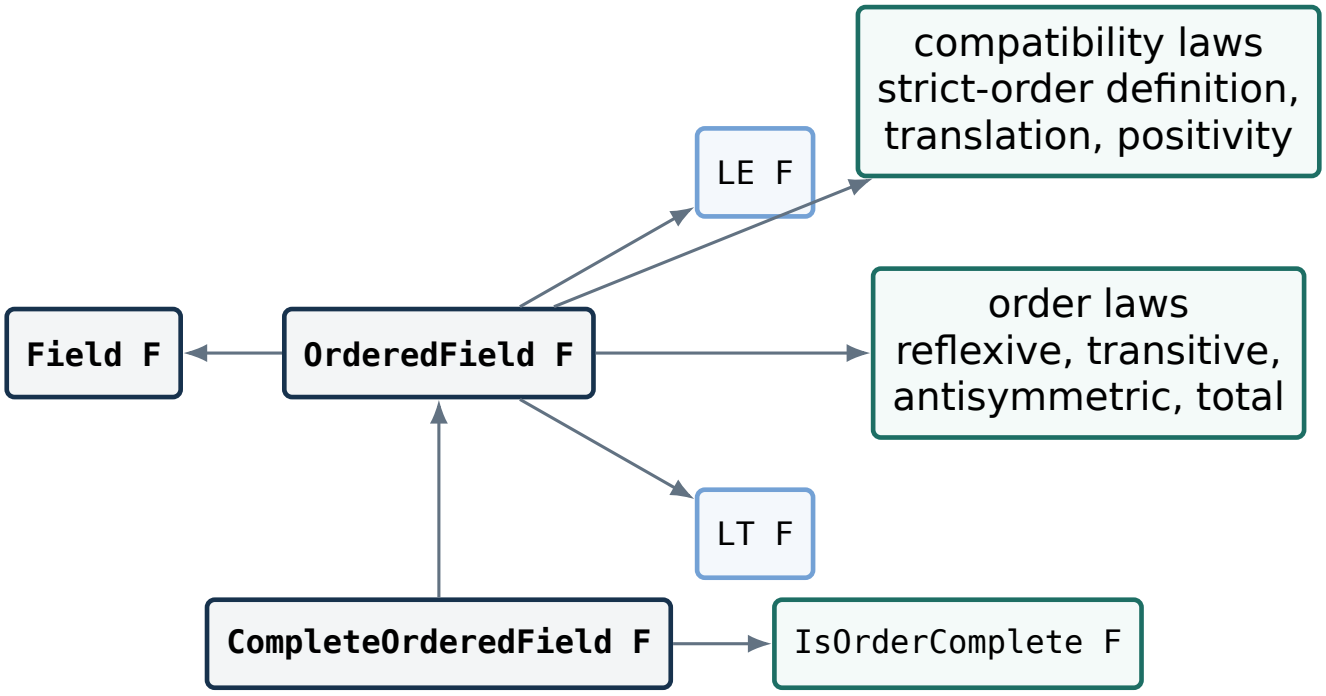


The arrows mean “can project as an instance,” not logical implication alone. If the local context contains `[Field F]`, a search for `[Add F]` or `[IsAddAssoc F]` can follow the generated parent projection.

8.2 Why no explicit `where` block?

All fields come from parents, so the class body is empty. Lean accepts the declaration without `where`. A concrete field construction will later need to provide values for the inherited parents, directly or through constructor syntax.

8.3 The full hierarchy



9 Derived data instances: nonemptiness, subtraction, division

The field bundle supplies enough primitive data to define additional standard interfaces.

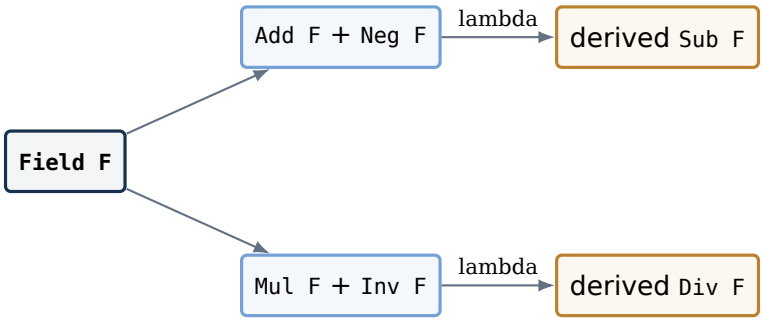
```
instance {F : Type u} [Field F] : Nonempty F :=
  ⟨0⟩
```

Once `[Field F]` is present, its generated `Zero F` parent makes `0 : F` available. That value witnesses that `F` is inhabited. `Nonempty F` is a proposition, so this records existence without choosing computational default behavior.

```
instance {F : Type u} [Field F] : Sub F :=
  ⟨fun a b => a + -b⟩

instance {F : Type u} [Field F] : Div F :=
  ⟨fun a b => a * b⁻¹⟩
```

`fun a b => ...` is lambda syntax. Subtraction and division are not new primitive field data in this design; they are computed from addition/negation and multiplication/inverse.



9.1 Definitional equality and `rfl`

```
theorem sub_eq_add_neg {F : Type u} [Field F] (a b : F) :
    a - b = a + -b :=
  rfl
```

After unfolding the selected `Sub F` instance, both sides are literally the same term. `rfl` proves reflexive equality; no algebraic axiom is used. This is *definitional* or computational equality, stronger than merely having a theorem relating two independently chosen operations. The division theorem is identical in syntax category and reasoning.

10 Predicates as sets and order completeness

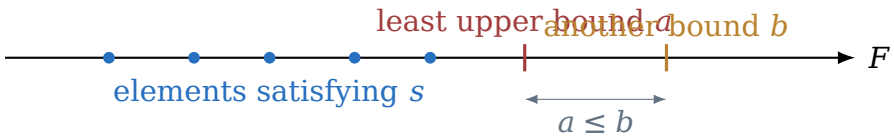
The file represents a set of `F`-elements by its membership predicate `s : F → Prop`. Saying `s x` means "`x` belongs to the set." This avoids requiring a separate set library.

```
def IsUpperBound {F : Type u} [LE F]
    (s : F → Prop) (a : F) : Prop :=
  ∀ x : F, s x → x ≤ a
```

This is a transparent definition: it introduces an abbreviation with computational content that Lean may unfold. It is not a class and is not searched for automatically.

```
def IsLeastUpperBound ... (s : F → Prop) (a : F) : Prop :=
  IsUpperBound s a ∧
  ∀ b : F, IsUpperBound s b → a ≤ b
```

The conjunction packages two proofs: `a` is an upper bound, and `a` is below every upper bound.



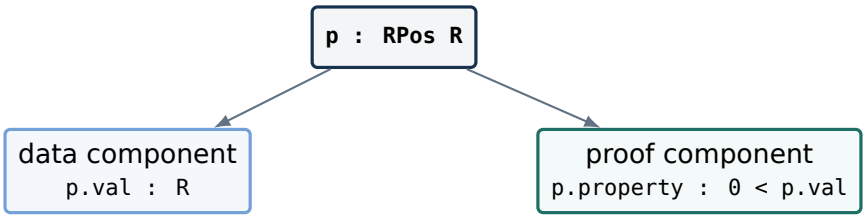
`IsOrderComplete` turns the least-upper-bound assertion into a reusable class law. The nested arrows expose its prerequisites:

1. choose any predicate `s`;
2. supply an existential witness proving it is nonempty;
3. supply an existential witness proving it has an upper bound;
4. receive an existentially packaged least upper bound.

11 Subtypes: values carrying proofs

```
def RPos (R : Type u) [Zero R] [LT R] : Type u :=
  { x : R // 0 < x }
```

The subtype syntax `\{ x : R // P x \}` denotes pairs consisting of a value `x : R` and a proof of `P x`. For a value `p : RPos R`, one can extract its underlying value and proof conceptually as `p.val` and `p.property`.



`RNeg` and `RNonneg` use exactly the same syntax category with different predicates. Because each is a `def`, it is a named subtype expression rather than a new opaque primitive type.

Watch this

The names begin with R, but the parameter can be any ordered carrier providing the requested instances. No real-number construction is required by these definitions.

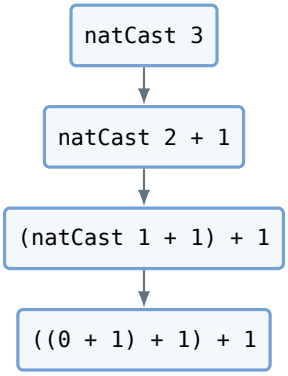
12 Recursive construction by pattern matching: natCast

```
def natCast {F : Type u} [Zero F] [One F] [Add F] : Nat → F
  | 0 => 0
  | n + 1 => natCast n + 1
```

The declaration defines a function from core Nat to F. Its equations are written directly as pattern-matching clauses.

- Input pattern 0: return the selected zero of F.
- Successor pattern n + 1: recursively cast n, then add the selected one of F.

Lean sees that the recursive call is made on the structurally smaller n, so it accepts termination automatically. No field laws are needed to define this function; only the three pieces of operational data used by its equations are required.



The result is a repeated-addition embedding candidate. Injectivity and preservation of addition or multiplication are intentionally future theorems, not consequences of the definition alone.

13 A dependent mixin: the Archimedean property

```
class ArchimedeanProperty (F : Type u) [OrderedField F] : Prop where
  archimedean : ∀ x : F, ∃ n : Nat, x < natCast n
```

This mixin depends on the entire ordered-field package rather than listing individual operations. That package provides LT F for < and also, through its Field F parent, the zero, one, and addition needed to infer the arguments of natCast.

The field says that for each x, a proof contains both a natural-number witness n and evidence of x < natCast n. The class remains separate even though the property is intended

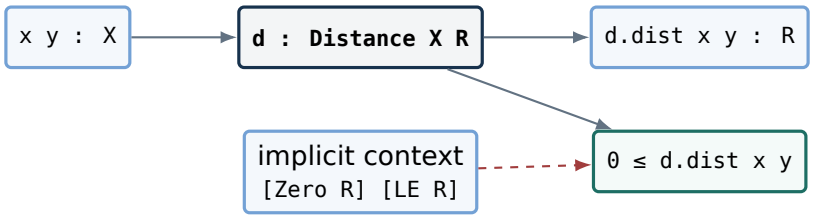
to be proved later from completeness: separation permits the same downstream theory to apply to ordered fields whose Archimedean property is given independently.

14 Ordinary structures: the revised `Distance`

```
structure Distance (X : Type u) (R : Type v) [Zero R] [LE R] where
  dist : X → X → R
  dist_nonneg : ∀ x y : X, 0 ≤ dist x y
```

This is an ordinary structure, not a class. A value `d : Distance X R` is passed explicitly unless some later declaration chooses to install one as an instance. It has two named fields:

1. `d.dist x y : R`, ordinary carrier-valued distance data;
2. `d.dist_nonneg x y`, a proof that this value is nonnegative.



Keeping `dist` carrier-valued is important: clients can use whatever algebraic operations `R` supports without repeatedly unwrapping a nonnegative subtype. The proof remains adjacent to the function and is available through a projection. This structure intentionally states only nonnegativity; symmetry, identity of indiscernibles, and the triangle inequality would belong to a stronger future metric structure.

15 Theorem declarations and proof styles

The smoke tests exercise three distinct proof mechanisms.

15.1 Direct term proof by projection

```
theorem field_smoke_test {F : Type u} [Field F] (a b c : F) :
    a + b + c = a + (b + c) :=
  IsAddAssoc.add_assoc a b c
```

After the colon comes the theorem's proposition. After `:=` comes a term of that type. The projection has a hidden `IsAddAssoc F` instance parameter, which inference obtains from the `Field F` parent.

15.2 Tactic proof

```
theorem field_ac_smoke_test ... :
    a + b + c = c + (b + a) := by
  ac_rfl
```

`by` opens tactic mode. `ac_rfl` normalizes expressions modulo associativity and commutativity. Its required standard witnesses are found through the custom bridge instances.

15.3 Definitional proof

The subtraction and division theorems use `rfl`, as discussed earlier. Their validity comes from unfolding definitions, not from the field-law mixins.

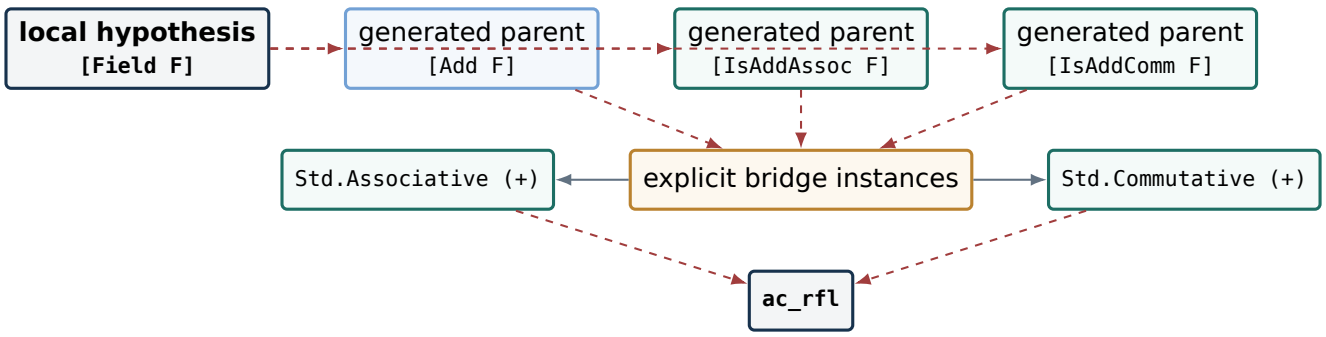
16 End-to-end inference traces

16.1 How `field_smoke_test` finds associativity

Elaboration trace

1. Local context contains `instField : Field F`.
2. The expression `a + b` creates a need for `Add F`.
3. Instance search uses the generated `Field.toAdd` parent projection.
4. `IsAddAssoc.add_assoc` creates a need for `IsAddAssoc F`.
5. Search uses the generated `Field.toIsAddAssoc` projection.
6. Applying the extracted universally quantified proof to `a`, `b`, and `c` produces exactly the target equality.

16.2 How `ac_rfl` crosses the custom/core boundary



The tactic never needs to know about `Field`. It requests standard algebraic properties of the particular operation. Instance inference connects those requests to the custom hierarchy.

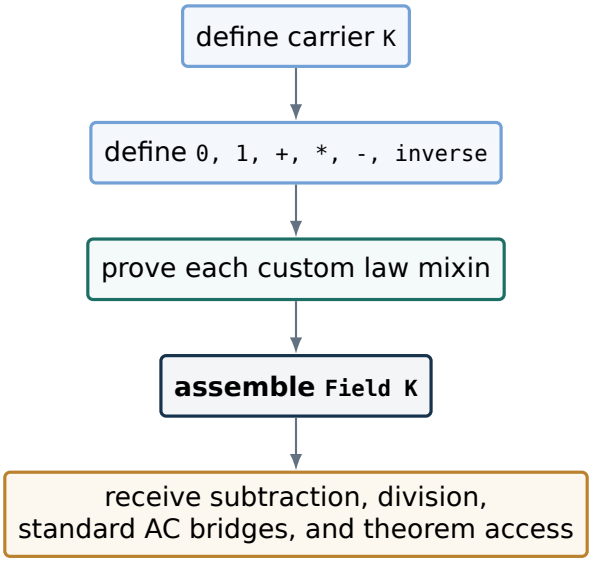
16.3 How x < natCast n elaborates

Elaboration trace

Inside ArchimedeanProperty, [OrderedField F] yields LT F directly and Field F transitively. The latter yields Zero F, One F, and Add F. These solve the implicit parameters of natCast; its result type is therefore F. Both sides of < now have type F, so the selected LT F relation applies.

17 Construction: what a future concrete field must provide

The present file defines interfaces, not a concrete instance. A later construction of a carrier K must eventually create a Field K. Conceptually the process is:



A schematic construction might look like the following; field names generated by extension can vary, so this is a conceptual template rather than promised copy-and-paste syntax:

```
instance : Field K where
  -- operation fields or parent instances
  -- proofs for zero_ne_one, add_assoc, ..., mul_add
```

Once the instance is registered, declarations with [Field K] can use it without receiving an explicit parameter. Local instances can override or specialize behavior, so a typeclass should represent data intended to be canonical in the relevant scope.

18 Syntax index for the entire file

This index groups every distinct syntactic mechanism. Repeated uses differ only in mathematical content.

Syntax category	Representative occurrence	Role
module doc comment	`/-! ... -/`	file-level generated documentation
declaration doc comment	`/-- ... -/`	attaches prose to the next declaration
namespace	`namespace aim.analysis`	qualifies all enclosed names
universe variables	`universe u v`	polymorphic levels for carrier types
explicit binder	`(X : Type u)`	caller-provided argument
implicit binder	`\{F : Type u\}`	inferred ordinary argument
instance binder	`[Add X]`	argument synthesized by typeclass search
class in `Prop`	`class IsAddAssoc ... : Prop`	searchable law package/mixin
class with parents	`class Field ... extends ...`	searchable combined data/law bundle
ordinary structure	`structure Distance ...`	explicit record with data and proof fields
definition	`def IsUpperBound ... :=`	transparent named term/type
recursive definition	`def natCast ... | ...`	terminating function by pattern matching
instance declaration	`instance ... : Sub F :=`	rule registered for typeclass inference
theorem declaration	`theorem ... : P := proof`	named proof term
universal quantifier	`∀ x : F, ...`	dependent function over every value
existential quantifier	`∃ x : F, ...`	witness paired with proof
logical connectives	`→ ∧ ∨ ↔ ¬`	functions and proof-combining types
operator notation	`+ * - / ≤ < ⁻¹`	syntax resolved through data instances
operator section	`(· + ·)`	anonymous binary function
lambda	`fun a b => ...`	explicit anonymous function
anonymous constructor	`⟨...⟩`	constructor inferred from expected type
subtype	`\{x : R // P x\}`	carrier value bundled with a proof
pattern clause	`| n + 1 => ...`	branch of a recursive definition
term proof	`IsAddAssoc.add_assoc a b c`	direct proof expression
tactic proof	`by ac_rfl`	tactic generates a proof term
reflexivity proof	`rfl`	closes definitional equality
type annotation	`(0 : X)`	fixes an otherwise ambiguous expected type
qualified name	`Std.Associative`	references declaration from another namespace

19 A disciplined way to read similar Lean files

When encountering a declaration in this library, use the following visual checklist:

1. **Find the result sort.** Is it a `Type`, a `Prop`, a theorem, or an instance?
2. **Classify every binder.** Parentheses, braces, and brackets imply different elaboration behavior.
3. **Inventory notation dependencies.** Every overloaded symbol requires a data instance.
4. **Separate data from laws.** Ask which declarations choose operations and which merely prove properties.
5. **Follow `extends` arrows.** These often explain where a missing instance comes from.
6. **Follow bridge instances.** A tactic may request a standard interface even when the local hierarchy uses custom names.

7. **Distinguish computation from theorem proving.** `rfl` reflects definitional reduction; a projection extracts an assumed law; a tactic constructs a new proof using available laws.

Big idea

The central wiring pattern of this file is:

$$\begin{aligned}\text{carrier} &\longrightarrow \text{operation instances} \longrightarrow \text{law mixins}\\ &\longrightarrow \text{bundled hierarchy} \longrightarrow \text{generated parent instances}\\ &\longrightarrow \text{theorems/tactics.}\end{aligned}$$

Once this chain is visible, most punctuation in the file stops looking ceremonial: each piece tells the elaborator where an input comes from and tells instance inference which edges it may traverse.

20 Compact glossary

Carrier
The underlying type whose elements are being studied, such as `F`.
Elaboration
Lean's conversion of convenient surface syntax into a fully typed term with implicit arguments and instances filled in.
Instance inference
Type-directed search for values whose parameters use square brackets.
Mixin
A small reusable class, usually proposition-valued here, expressing one law.
Parent projection
A generated function/instance extracting an extended class from a larger class.
Definitional equality
Equality visible by computation/unfolding, provable by `rfl`.
Propositional equality
The equality type `a = b`; it may require an actual proof using laws.
Subtype
A value of a carrier paired with proof that it satisfies a predicate.
Predicate-as-set
A function `F → Prop`; membership is function application.
Canonical data
Data intended to be found automatically as a typeclass instance.